AIC

2026 年第八届全球校园人工智能算法精英大赛

算法主题赛

AI+开源

技术报告

团队名称：个人组队

团队编号：待填写

作品名称：ReproLens 开源复现审计

参赛方向：AI 开发工具与开源协作

日期：2026 年 10 月 29 日

ReproLens 开源复现审计技术报告

Evidence-constrained reproduction audit for open-source AI repositories

报告摘要

- ReproLens 面向公开 AI/ML 论文代码仓库，先收集文件位置、原文和可执行线索，再把 README 声明映射到固定检查项，输出可追溯的复现体检报告。
- 系统不以关键词命中代替证据：依赖清单不等于版本锁定，数据链接不等于数据脚本，通用 LICENSE 不等于数据或模型许可；没有对应证据时显示缺口或 N/A。
- 最终样例 HiHR 为视觉-语言行人重识别项目，审计识别为 ml_project，LLM 特有检查为 N/A；报告得分 79.0/100，证据覆盖率 14/17（82.4%）。
- 作品同时提供 Web、命令行、JSON、GitHub Action 和飞书通知接口；Webhook 只保存在服务器环境变量中，前端不暴露。

一、作品概述

1.1 背景与问题

论文作者公开代码并不等于项目可以直接复现。复现者需要在 README、目录结构、依赖文件、配置、脚本、数据集和模型来源之间交叉核对；这些证据通常分散，且容易被“看起来提到了”误当成“已经有证据”。在大语言模型、RAG 和 Agent 项目中，模型版本、Prompt、采样参数、检索索引与调用轨迹又增加了新的不确定性。

人工阅读仓库可以发现问题，但难以稳定复用：同一份 README 在不同评审者眼中可能得到不同判断，GitHub issue、CI 和飞书群之间也缺少统一的结构化结果。ReproLens 将这项工作拆成可审计的证据检查，并把发现、缺口、整改建议和外部通知串成一条流程。

1.2 目标、方向与作品形态

作品目标是：给定公开 GitHub 仓库，自动生成一份可以回溯到文件和原文的复现体检报告；对不适用项明确标记 N/A，对缺口给出最小整改建议；把结果输出为机器可读 JSON，并通过 Webhook 接入 CI、GitHub Actions 或飞书群。

参赛方向为“AI 开发工具与开源协作”。作品形态包括 Web 页面、Python 命令行、GitHub Action、HTML/Markdown 报告、JSON 外部接口和飞书通知适配器。当前版本优先支持 Python 为主的 AI/ML 论文代码仓库，非 Python、模型无关 SDK、私有资源和超大仓库可能只覆盖通用检查。

1.3 核心亮点

- 证据约束：每条结论必须对应文件、行、原文或实际执行记录，不能用无关线索替代。
- 缺口直述：主要发现直接写“缺少什么”，不把“是否……”当作问题陈述。
- 类型判断与应用性：LLM/RAG/Agent、普通 ML 项目和模型无关 SDK 使用不同的适用规则，无法判断时显示 N/A。
- 本地 AI 参与：本地开源语义模型只做 README 声明到检查项的候选映射，不直接决定分数，失败回退到规则模式。
- AutoFix 复测：候选文件只在隔离副本中加入，逐个重扫，只保留能提高评分的修改。

二、需求分析与总体方案

2.1 目标用户与需求分析

目标用户包括准备复现论文的学生和研究人员、维护公开 AI 项目的开发者、需要快速了解仓库质量的评审者，以及希望在 CI 中设置质量门槛的团队。现有做法的共同问题是：人工检查缺少统一标准，报告只有自然语言结论而无法回到原文，通用大模型可以生成解释但容易把不存在的能力说得像真的。

因此需求优先级依次为：证据可追溯、结论可复核、适用范围明确、整改可执行、结果可接入。只会生成一段“看起来像报告”的文字，不足以解决复现问题。

2.2 总体方案

ReproLens 采用“静态证据优先、AI 受限参与、规则负责判定”的结构。公共仓库被固定到具体 commit 后，扫描器和脚本分析器提取证据；25 个检查项按环境、数据、模型、实验、文档和 LLM 特有风险六个维度计算；声明矩阵把 README 原句映射到检查项；最后生成报告、AutoFix 提案、JSON 和外部通知。

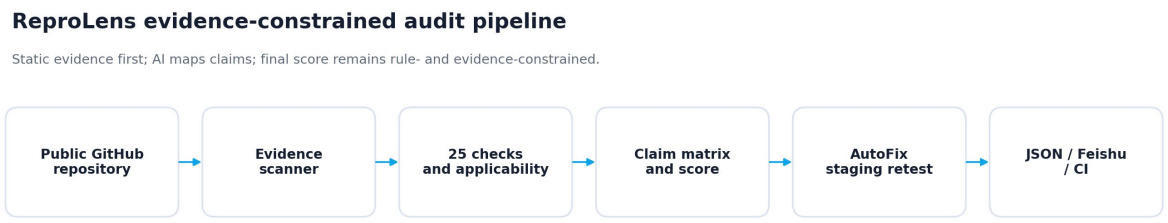


图 1 ReproLens 证据约束审计流水线

输入	处理	输出
公开 GitHub 仓库或 GitHub Pages 链接	固定 commit、静态扫描、脚本结构分析、声明映射	报告、证据附录、声明矩阵、score.json
分数门槛与仓库白名单	规则检查、可应用性判断、质量门禁	通过/未通过、飞书通知、CI 退出码
未通过检查项	隔离 staging 复测 AutoFix 候选文件	proposal、patch、autofix.json

2.3 可行性与成本

作品不训练大语言模型，也不依赖商业模型 API。默认使用 Python 标准库、轻量扫描规则和本地语义模型；网页审计在服务器 CPU 上完成，AutoFix 只在隔离副本中执行静态复测。这样的成本结构适合个人组队在比赛周期内完成，并且不会因为外部 API 额度或密钥失效导致演示中断。

三、AI 技术与开源及第三方资源选择

3.1 AI 在作品中的具体作用

AI 的职责是“把 README 声明映射到候选检查项”，不是自由生成结论。系统先由规则扫描器得到候选检查项，再由本地语义模型给出候选映射、置信度和原因；映射结果仍要经过检查项目录、证据编号和适用性校验。模型不能创建不存在的证据编号，也不能直接修改总分。

本地模型 ReproLens-SmallSemantic-v1 在独立的 30 条声明评测中达到 Top-1 96.7%、Top-2 96.7%、覆盖率 100%。这些数字用于验证“声明映射”这个局部能力，不代表整个项目的复现成功。若本地模型不可用，系统回退到确定性规则，保证结果仍然可以生成。

3.2 开源及第三方资源选择

项目自身代码、检查规则、报告模板和文档以 MIT License 开源。公开仓库只作为审计对象读取，不复制其数据、模型权重或完整源码；飞书只作为通知渠道，Webhook 保存在服务器端环境变量中，不进入仓库、前端或 JSON 输出。

资源	版本/来源	许可证/授权	使用方式	关键义务或限制
ReproLens 自有代码	本仓库 main 分支	MIT	Web/CLI/Action/JSON	保留版权和许可声明
ReproLens-SmallSemantic-v1	本仓库模型卡	MIT	本地 CPU 语义映射	不直接决定分数；失败回退规则
Python	3.11+ 运行时	PSF License	执行扫描与 Web 服务	遵循 Python 许可
公开 GitHub 仓库	用户输入	由各仓库决定	只读静态审计	不复制数据、不执行第三方代码
飞书机器人	外部通知服务	服务条款	服务器端 Webhook	仅白名单仓库；地址不入库

3.3 自主开发与权利边界

检查规则、证据类型、缺口表达、报告页面、AutoFix 隔离复测、JSON 契约、测试和文档由个人组队完成。AI 编程助手用于代码草拟、问题排查和文档整理，所有生成代码、测试结果和报告结论均经过人工复核；最终产品运行时不调用商业大模型 API。第三方资源只按许可证和公开接口使用，不包含学校、实验室或其他权利主体的未授权材料。

3.4 融合方式

作品不是把通用大模型接在网页后面，而是把 AI 限定在“语义候选映射”这一层：扫描器、规则、证据校验、评分和 AutoFix 仍由 ReproLens 自身实现。AI 的产物必须落到固定检查项和有来源的证据上，因此即使模型判断错误，也不会自动生成一个看似完整但不可核验的结论。

四、设计与实现过程

4.1 核心功能与操作流程

用户在网页输入公开仓库地址，系统先解析并固定 commit，然后扫描文本文件、脚本、配置和依赖清单。扫描结果被转换为带类型、文件位置、行号和原文摘录的证据；随后执行 25 个检查项，按项目类型决定是否适用，并生成证据覆盖率、声明验证矩阵和最小整改计划。最后把结果写入 HTML、Markdown、JSON 和 AutoFix 目录。

Web 页面只返回报告链接和通知状态，不返回 Webhook。服务器在审计结束后读取同一份结构化结果，生成 Feishu 文本消息，并通过受保护的环境变量发送；白名单和 60 秒频率限制用于避免公开访问者刷屏。

4.2 关键实现

证据扫描器将“依赖清单”“版本锁定文件”“数据链接”“数据脚本”“模型来源”“模型许可”“执行探测”等类型分开，避免把相关线索合并成同一条件。检查规则在部分满足时只附上实际支持该条件的证据；缺口项不强行附上无关编号。

声明矩阵将 README 原句、验证状态、检查依据、缺口和支持证据放在同一行。HTML 表格固定首行和前两列，证据编号带中文类型和原文位置。AutoFix 先生成候选文件，再复制到 staging 目录逐个加入并重新扫描；只有分数提高的候选才进入 proposal 目录，原始仓库内容指纹前后保持一致。

4.3 团队分工与过程记录

本项目为个人组队。个人负责需求拆解、规则设计、扫描器与 Web 实现、测试、部署、文档和演示；使用 AI 编程助手辅助代码草拟、错误定位和文档组织，但每个修改都经过本地测试、真实仓库复核和人工判断。开发过程按“基础审计 → 证据相关性修正 → 领域适用性修正 → Web/飞书集成 → HiHR 最终样例验证”迭代。

五、成果展示与效果验证

5.1 最终样例：HiHR 仓库

最终样例使用 HiHR 论文代码仓库：https://github.com/YangQiWei3/HiHR。README 显示该论文研究 Aerial-Ground Person Re-Identification，使用 CLIP-ViT、文本提示、多粒度特征融合和双曲层级学习。它不是生成式大语言模型，而是视觉-语言行人重识别项目。

ReproLens 将其识别为 ml_project，LLM 特有风险维度为 N/A。这个结果也验证了此前对 faiss 依赖和 logger 中 run_id 的误判修复：依赖库和通用日志字段不能单独作为 RAG 或 Agent 证据。

指标	结果
扫描文件	242
总分	79.0 / 100
就绪度	Partially Ready
证据覆盖率	14/17（82.4%）
README 声明	6 条
LLM 特有风险	N/A
AutoFix	79.0 → 84.9（+5.9）



图 2 HiHR 网页体检报告总览（本地语义模型模式）

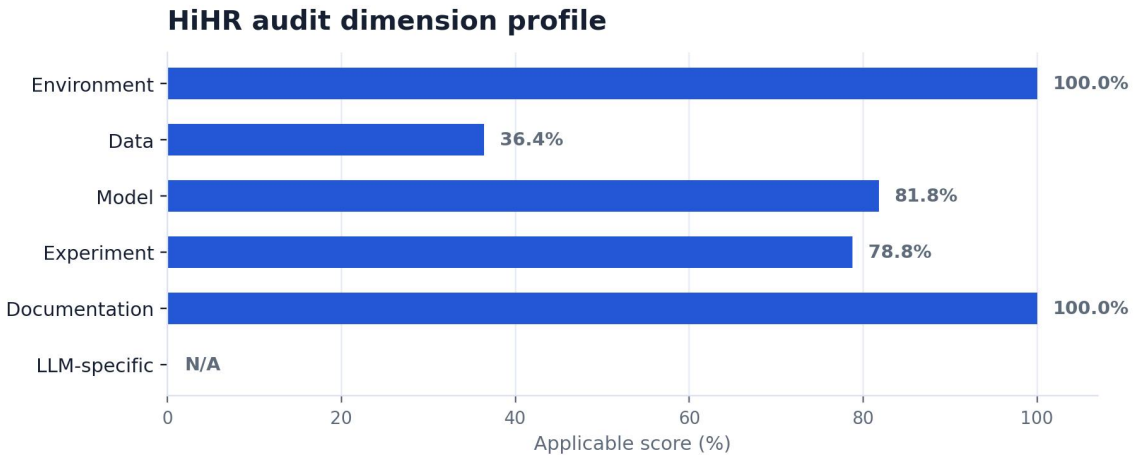


图 3 HiHR 六维检查结果

5.2 主要发现与 AutoFix 结果

编号	检查项	状态	结论
F-001	DATA-02 数据下载或预处理脚本	HIGH	缺少数据下载或预处理脚本
F-002	DATA-03 数据集卡或数据许可	MEDIUM	缺少数据集卡或明确的数据许可说明
F-003	MODEL-02 模型卡或版本说明	MEDIUM	有模型来源和线索，但仍缺完整模型卡/版本说明
F-004	EXP-05 预期结果和指标	HIGH	缺少预期结果和指标

AutoFix 将 79.0 分提升到 84.9 分，接受的候选文件是 data/download.sh 和 dataset_card.md；model_card.md、configs/llm_metadata.yaml、eval/README.md 因没有提高评分被拒绝。接受文件中的 TODO 仍需要维护者补充，系统不会把模板当成完整证据。原仓库内容指纹在 AutoFix 前后保持不变。

5.3 JSON、飞书与执行证据

每次审计同时生成 score.json，包含 source_url、source_commit、total_score、readiness、evidence_coverage、claim_matrix、findings 和 criteria。服务器端通知适配器从 JSON 提取精简摘要，转换成飞书文本消息，再 POST 到受保护的 Webhook；浏览器只看到“飞书通知：已发送”等状态。

执行证据只记录实际运行过的安全探测，例如 python-syntax；它证明探测执行过，不代表完整复现成功。当前默认不执行第三方仓库代码、不安装依赖，--execute 只建议在可信隔离副本中使用。

5.4 验证方法

- 功能测试：规则、扫描器、报告、声明矩阵、AutoFix、Web API、JSON 和通知适配器共 90 项自动化测试全部通过。
- 真实仓库验证：以 HiHR 的 242 个文件为最终样例，报告生成、证据覆盖率、N/A、AutoFix 与 JSON 输出均可复现。
- 外部通知验证：服务器端测试返回 notification_status=sent；命令行测试返回 Webhook: delivered (200)。
- 安全验证：Webhook 不出现在前端代码或 API 响应；白名单限制为 Cat-Zero/ViSA、czkPK/reprolens、YangQiWei3/HiHR，同一仓库通知间隔 60 秒。

六、创新性与开放价值

6.1 创新性

ReproLens 的主要创新不是再生成一段报告，而是把“证据相关性”作为评分前置条件。依赖清单、数据链接、项目 LICENSE、faiss 依赖和 run_id 这些容易被通用评审误用的线索，被拆成有明确边界的证据类型；没有直接支持时，系统显示 N/A 或缺口，而不是为了分数附上无关编号。

第二个创新点是领域判断的可解释性：系统根据模型 API、Prompt、采样参数、RAG/Agent 结构和配置证据区分 LLM/RAG/Agent 与普通 ML 项目；HiHR 这种 CLIP 视觉-语言项目不会被简单地判成 LLM。

第三个创新点是 AutoFix 的隔离复测：候选修改必须经过重新扫描并提高评分才会被保留，原始仓库不变，整改提案与补丁同时输出。

6.2 开放成果与复用方式

作品以 MIT License 开源，提供 Web 页面、CLI、GitHub Action、JSON 外部接口、AutoFix 报告和使用文档。其他研究者可以复用同一套 score.json 接入 CI、飞书或内部质量平台，也可以增加新的检查规则和领域适配器。

6.3 维护与后续计划

后续将继续扩大跨语言、跨框架和非 Python 项目的规则覆盖，增加更多真实仓库回归样例，完善本地语义模型的校准和人工覆盖机制，并把飞书通知扩展为可配置的团队工作流。

七、合规、安全与风险说明

7.1 开源与第三方许可

自有代码和文档采用 MIT License。Python 运行时遵循 PSF License；被审计的 GitHub 仓库、数据集和模型权重仍由各自权利人授权，系统只读取公开内容，不复制或再分发数据、权重和完整源码。飞书机器人属于第三方服务，使用时遵守其服务条款。

7.2 数据、隐私与通知安全

网页只接收公开仓库地址，不要求姓名、学校或个人信息。飞书 Webhook 保存在服务器 /etc/reprolens/notify.env，文件权限为 600，不进入前端、JSON 或 GitHub 仓库；白名单和频率限制用于避免公开访问者滥用。审计不向第三方模型发送仓库内容。

7.3 AI 输出与内容安全

本地语义模型只给出声明到检查项的候选映射，不直接生成分数；映射结果要经过检查项目录、证据编号和适用性校验。模型输出错误时回退到规则模式，报告中的 N/A 和缺口仍由规则和证据决定。AutoFix 生成文件中的 TODO 不会被当作已验证证据，发布前需要维护者人工补充和复核。

7.4 技术限制

本报告是静态复现就绪度分析，不代表项目已经实际运行成功；默认不执行第三方代码；大型仓库可能跳过 AutoFix；当前优先支持 Python 为主的 AI/ML 项目；模型判断和语义映射仍需要人工复核，不能把分数当作论文质量的唯一结论。

八、总结与展望

ReproLens 已完成从公开仓库到证据报告、声明矩阵、AutoFix 提案、JSON 接口和飞书通知的完整链路。HiHR 最终样例表明：系统可以识别视觉-语言项目与 LLM 项目的差异，能够在缺少直接证据时保留缺口而不是强行加分，也能通过隔离副本验证整改提案。

下一阶段将优先补充非 Python 项目和模型无关 SDK 的规则，扩展真实仓库基准，完善人工覆盖和证据审核界面，并把通知、质量门槛和团队协作做成可配置的开放工作流。

九、附录

9.1 在线地址与仓库

- GitHub 仓库：https://github.com/czkPK/reprolens
- 网页版：http://182.92.232.168:8080/
- HiHR 样例报告：http://182.92.232.168:8080/reports/20261029-201000-hihr03/report.html
- HiHR 源码：https://github.com/YangQiWei3/HiHR
- JSON 样例：http://182.92.232.168:8080/reports/20261029-111143-203b6044/score.json

9.2 验证记录

验证项	记录
自动化测试	90 项通过
HiHR 扫描	242 个文件，report.html / claims.html / autofix.html / score.json 均生成
HiHR 评分	79.0 / 100；Partially Ready；证据覆盖率 14/17
HiHR AutoFix	79.0 → 84.9；接受 data/download.sh、dataset_card.md
飞书通知	notification_status=sent；命令行 Webhook: delivered (200)
原始仓库保护	AutoFix 前后内容指纹一致；默认不执行第三方代码

9.3 开源及第三方资源使用清单

资源名称及类型	版本/来源	许可证/授权	使用方式	关键义务/限制	修改或开发内容	合规状态
ReproLens 代码	本仓库	MIT	Web/CLI/Action	保留声明	规则、UI、测试、文档	合规
本地语义模型	本仓库	MIT	CPU 声明映射	不决定分数	模型卡与回退逻辑	合规
Python	3.11+	PSF	运行时	遵循许可	适配 Web/CLI	合规
公开论文仓库	用户输入	各仓库许可	只读审计	不复制数据/权重	证据提取	合规
飞书机器人	第三方服务	服务条款	Webhook 通知	白名单/频率限制	消息适配器	合规

说明：报告中的图片来自 ReproLens 实际运行页面和根据同一份审计结果生成的图表；不含飞书 Webhook、API Key 或其他敏感凭据。